Typescript

Class ****

A class in TypeScript is a template for creating objects, containing properties and methods.

```
class Animal {
    private name: string;
    protected age: number;
    public constructor(name: string, age: number) {
        this.name = name;
        this.age = age;
    }
    public move(distance: number): void {
        console.log(`${this.name} moved ${distance}m.`);
    }
}
```

Class Important Notes

I. **Access Modifiers**:

- public: Accessible from anywhere
- private: Accessible only within the class
- protected: Accessible within the class and its subclasses

II. **Constructor**:

- Called when creating a new instance
- Can have parameters
- Must call super() in child classes

III. **Inheritance**:

- Use extends for inheritance
- Can override methods
- Can use super to call parent class methods

IV. **Static Members**:

- Belong to the class, not instances
- Accessed through class name
- Cannot be accessed from instances

Decorator

Decorators are a design pattern that allows adding new behavior to an object without modifying its code.

I. **Class Decorator**
II. **Method Decorator**
III. **Property Decorator**
IV. **Parameter Decorator**
V. **Decorator Factory**

Notes:

I. Decorator runs when class is defined, does not run when creating instance
II. Decorators are executed from top to bottom
III. Enable experimentalDecorators in tsconfig.json

Class Decorator

A Class Decorator is a function that is called when a class is defined, not when it is instantiated.

```
// Decorator function
function log(target: Function) {
    console.log('Class decorated:', target);
}

// Using decorator
@log
class User {
    name: string;
    constructor(name: string) {
        this.name = name;
    }
}
```

Class Decorator

```typescript
// Singleton Decorator
function singleton<T extends { new (...args: any[]): any }>(constructor: T) {
    let instance: any;

    return class extends constructor {
        constructor(...args: any[]) {
            super(...args);
            if (instance) {
                return instance;
            }
            instance = this;
        }
    };
}
```

```typescript
@singleton
class Database {
    private connection: string;

    constructor() {
        this.connection = 'Connected to database';
        console.log('Database initialized');
    }

    query(sql: string) {
        console.log(`Executing query: ${sql}`);
    }
}
// Test
const db1 = new Database();
const db2 = new Database();
console.log(db1 === db2); // true - chỉ tạo một instance
```

Function Decorator

- A **function decorator** is a function used to “decorate” (enhance, modify, or add behavior to) a method within a class.
- It is placed directly above the method using the @ symbol.
- When the class is defined, the decorator receives information about the method and can alter, log, validate, or modify the method’s behavior.

Parameters of a Function Decorator

- **target**: The prototype of the class (if it’s an instance method) or the constructor (if it’s a static method).
- **propertyKey**: The name of the method (as a string).
- **descriptor**: An object describing the method, allowing you to change how the method behaves (e.g., override, add logic before/after calling the original method).

Important Notes

- Function decorators only apply to methods within a class, not to standalone functions outside a class.
- Decorators run only once when the class is defined, not every time the method is called.

```
function log(target: any, propertyKey: string, descriptor: PropertyDescriptor) {
    console.log('Decorating:', propertyKey);
}

class User {
    @log
    sayHello() {
        console.log('Hello!');
    }
}
```

Function Decorator

```typescript
function log(target: any, propertyKey: string, descriptor: PropertyDescriptor) {
    const originalMethod = descriptor.value;
    descriptor.value = function (...args: any[]) {
        console.log(`Method ${propertyKey} called with parameters:`, args);
        const result = originalMethod.apply(this, args);
        console.log(`Returned result:`, result);
        return result;
    };
}

class Calculator {
    @log
    add(a: number, b: number) {
        return a + b;
    }
}

const calc = new Calculator();
calc.add(2, 3);
// Log output:
// Method add called with parameters: [2, 3]
// Returned result: 5
```

Property Decorator

- A **property decorator** is a function used to “decorate” (enhance, modify, or add behavior to) a property within a class.
- It is placed directly above the property using the @ symbol.
- When the class is defined, the decorator receives information about the property and can alter, log, validate, or modify the property’s behavior.

Parameters of a Property Decorator

- **target**: The prototype of the class (if it’s an instance property) or the constructor (if it’s a static property).
- **propertyKey**: The name of the property (as a string).

Important Notes

- Property decorators only apply to properties within a class, not to standalone variables outside a class.
- Decorators run only once when the class is defined, not every time the property is accessed.

```
function log(target: any, propertyKey: string) {
    console.log('Decorating property:', propertyKey);
}

class User {
    @log
    name: string;

    constructor(name: string) {
        this.name = name;
    }
}
```

Property Decorator

```typescript
function log(target: any, propertyKey: string) {
    let value: any;

    // Create getter and setter for the property
    const getter = function() {
        console.log(`Getting value of ${propertyKey}:`, value);
        return value;
    };

    const setter = function(newVal: any) {
        console.log(`Setting value of ${propertyKey} to:`, newVal);
        value = newVal;
    };

    // Replace the property with getter and setter
    Object.defineProperty(target, propertyKey, {
        get: getter,
        set: setter,
        enumerable: true,
        configurable: true
    });
}
```

```typescript
class User {
    @log
    name: string;

    constructor(name: string) {
        this.name = name;
    }
}

const user = new User('John');
console.log(user.name); // Calls getter
user.name = 'Jane'; // Calls setter


// Getting value of name: John
// Setting value of name to: Jane
```

What is module?

- In JavaScript, a module is a unit of code (a file or a set of files) that encapsulates variables, functions, classes, etc., and only exposes (exports) whatever you want to share externally. Splitting your code into modules helps you:

- Modules are just clusters of code, but it should
  - highly self-contained with distinct functionality
  - can be shuffled, removed, or added as necessary
  - aims to lessen the dependencies on parts of the codebase as much as possible

What is module look like?

1. Single responsibility
2. Independency
3. Encapsulation
4. Small

Why?

1. Easy to read

2. Easy to debug

3. Easy to maintain

4. Reusability

5. Avoid namespace pollution

How to make module in typescript?

There are two main module systems in JS today:

I. **CommonJS (CJS)**

II. **ES Module (ESM)**

Export / Import

```
//Named Export
export const add = (a: number, b: number) => a + b;
export const subtract = (a: number, b: number) => a - b;

//Default Export
export default class Calculator {
    add(a: number, b: number) {
        return a + b;
    }
}
```

```
//Named Import
import { add, subtract } from './file';

//Default Import
import Calculator from './calculator';
```

Math Module Example

```typescript
// Named exports
export const add = (a: number, b: number): number => {
    return a + b;
};

export const subtract = (a: number, b: number): number => {
    return a - b;
};

export const multiply = (a: number, b: number): number => {
    return a * b;
};

export const divide = (a: number, b: number): number => {
    if (b === 0) {
        throw new Error('Cannot divide by zero');
    }
    return a / b;
};
```

```typescript
import { add, subtract, multiply, divide } from './math';
console.log('Using named exports:');
console.log('Add:', add(5, 3));          // 8
console.log('Subtract:', subtract(5, 3)); // 2
console.log('Multiply:', multiply(5, 3)); // 15
console.log('Divide:', divide(6, 2));    // 3
```

Math Module Example

```
export default class Calculator {

    add(a: number, b: number): number {
        return a + b;
    }

    subtract(a: number, b: number): number {
        return a - b;
    }
}
```

```
import Calculator from './math';

const calculator = new Calculator();

const result1 = calculator.add(10, 5);
const result1 = calculator.add(10, 5);
console.log('Add result:', result1);  // 15

const result2 = calculator.subtract(10, 5);
console.log('Subtract result:', result2);  // 5
```

What is Namespace

- A **namespace** is a way to organize code into logical groups, helping to avoid naming conflicts.
- Similar to modules, but namespaces can contain multiple files within the same namespace.

```
namespace MathOperations {
    export function add(a: number, b: number): number {
        return a + b;
    }

    export function subtract(a: number, b: number): number {
        return a - b;
    }
}

// Usage
console.log(MathOperations.add(5, 3)); // 8
```

Namespace with Multiple Files

- Can contain multiple files in the same namespace
- Uses `/// <reference>` to link files
- Careful circular reference

```
// shapes.ts
namespace Geometry {
    export class Rectangle {
        constructor(public width: number, public height:
number) {}

        getArea(): number {
            return this.width * this.height;
        }
    }
}
//utils.ts
namespace Geometry {
    export function calculatePerimeter(rect: Rectangle):
number {
        return 2 * (rect.width + rect.height);
    }
}
//main.ts
/// <reference path="shapes.ts" />
/// <reference path="utils.ts" />

const rect = new Geometry.Rectangle(5, 3);
console.log(rect.getArea()); // 15
console.log(Geometry.calculatePerimeter(rect)); // 16
```

Nested Namespace

```
namespace Geometry {
    export namespace Shapes {
        export class Rectangle {
            constructor(public width: number, public height:
number) {}

            getArea(): number {
                return this.width * this.height;
            }
        }

        export class Circle {
            constructor(public radius: number) {}

            getArea(): number {
                return Math.PI * this.radius * this.radius;
            }
        }
    }

    export namespace Utils {
        export function calculatePerimeter(shape:
Shapes.Rectangle): number {
            return 2 * (shape.width + shape.height);
        }
    }
}
```

```
// Usage
const rect = new Geometry.Shapes.Rectangle(5, 3);
console.log(rect.getArea()); // 15
console.log(Geometry.Utils.calculatePerimeter(rect));
// 16
```

But

About the "module" keyword:

- We shouldn't use the "module" keyword in TypeScript
- The "module" keyword has been deprecated (no longer recommended)
- Instead, we should use the `import/export` syntax of ES modules

About “namespace” keyword:

- It's true that namespace is more powerful for code organization
- Can define a namespace across multiple files
- Can use `--outFile` to combine files into a single file

`To summarize, when writing new TypeScript code, you should use ES modules instead of namespace.`

What is Declare

- `declare` is a keyword in TypeScript used to declare data types for variables, functions, classes without needing to define their implementation.
- Commonly used when working with external JavaScript code or third-party libraries.
- Helps TypeScript understand the data types of external components
- Can declare variables, functions, classes, interfaces, modules
- Should separate declarations into dedicated .d.ts files

```
//variables
declare const API_KEY: string;
declare let globalVariable: number;
//function
declare function fetchData(url: string): Promise<any>;
declare function calculateTotal(price: number, tax: number):
number;
//Class
declare class ExternalLibrary {
    constructor(config: any);
    method1(): void;
    method2(param: string): number;
}
```